



1η Εργασία στο Μάθημα Ασφάλεια Συστημάτων

Πίνακας περιεχομένων

1η Εργασία στο Μάθημα Ασφάλεια Συστημάτων	1
ΜΕΡΟΣ Α: Υπολογισμός Hash και Έλεγχος Ακεραιότητας	1
Βήματα Υλοποίησης	1
ΜΕΡΟΣ Β: Υπολογισμός και Ανάλυση Εντροπίας	2
Βήματα Υλοποίησης	2
ΜΕΡΟΣ Γ: Αυθεντικοποίηση Δύο Παραγόντων (2FA).....	2
Βήματα Υλοποίησης	2
Μέρος Δ:Ενσωμάτωση Salt	3
Βήματα Υλοποίησης	3
Διαδραστικό Μενού	3
Παραδοτέα	3
Παράρτημα-Υποδείξεις.....	4
Ανάγνωση Αρχείου	4
Υπολογισμός Hash	4
Αποθήκευση Hash	4
Έλεγχος Ακεραιότητας.....	4
Υπολογισμός Εντροπίας	5
Διαδραστικό Μενού	5

ΜΕΡΟΣ Α: Υπολογισμός Hash και Έλεγχος Ακεραιότητας

Στο πρώτο μέρος της άσκησης, καλείστε να υλοποιήσετε τη βασική λειτουργικότητα υπολογισμού συναρτήσεων κατακερματισμού και ελέγχου ακεραιότητας αρχείων.

Βήματα Υλοποίησης

- Αρχικά, το πρόγραμμα θα πρέπει να ζητά από τον χρήστη το όνομα ενός αρχείου και να το διαβάζει σε δυαδική μορφή (binary mode), ώστε να μπορεί να επεξεργαστεί οποιονδήποτε τύπο αρχείου.
- Στη συνέχεια, θα πρέπει να υλοποιηθεί ο υπολογισμός κρυπτογραφικών συναρτήσεων κατακερματισμού. Καλείστε να χρησιμοποιήσετε αλγορίθμους όπως MD5, SHA-1 ή SHA-256, καθώς και SHA-3 (Keccak). Το πρόγραμμα θα πρέπει να δίνει τη δυνατότητα επιλογής ενός συγκεκριμένου αλγορίθμου ή εκτέλεσης όλων.



- Έπειτα, το πρόγραμμα θα εμφανίζει τα αποτελέσματα των hashes στην οθόνη με κατανοητό τρόπο.
- Στο επόμενο βήμα, θα πρέπει να αποθηκεύεται το hash σε αρχείο (π.χ. filename.hash), ώστε να χρησιμοποιηθεί για μελλοντικό έλεγχο.
- Τέλος, θα υλοποιηθεί ο έλεγχος ακεραιότητας. Το πρόγραμμα θα επαναυπολογίζει το hash του ίδιου αρχείου και θα το συγκρίνει με το αποθηκευμένο. Αν τα hashes είναι ίδια, θα εμφανίζεται μήνυμα ότι το αρχείο δεν έχει τροποποιηθεί, ενώ αν διαφέρουν, θα εμφανίζεται μήνυμα ότι το αρχείο έχει αλλοιωθεί.

ΜΕΡΟΣ Β: Υπολογισμός και Ανάλυση Εντροπίας

Στο δεύτερο μέρος, καλείστε να υλοποιήσετε τον υπολογισμό της εντροπίας των δεδομένων, βασισμένο στην έννοια της Shannon Entropy.

Το αρχείο θα θεωρείται ως ακολουθία bytes και ο δειγματικός χώρος περιλαμβάνει όλες τις δυνατές τιμές byte (0 έως 255).

Βήματα Υλοποίησης

- Αρχικά, το πρόγραμμα θα διαβάζει το αρχείο σε binary μορφή.
- Στη συνέχεια, θα υπολογίζει πόσες φορές εμφανίζεται κάθε τιμή byte (0–255) μέσα στο αρχείο.
- Έπειτα, οι συχνότητες αυτές θα μετατρέπονται σε πιθανότητες, διαιρώντας κάθε πλήθος εμφανίσεων με το συνολικό πλήθος bytes του αρχείου.
- Στο επόμενο βήμα, θα εφαρμόζεται ο μαθηματικός τύπος της εντροπίας:

$$H = - \sum_{i=1}^{256} p(x_i) \log_2 p(x_i)$$

- Τέλος, το πρόγραμμα θα εμφανίζει την τιμή της εντροπίας. Τιμές κοντά στο 8 υποδηλώνουν υψηλή τυχαιότητα (π.χ. κρυπτογραφημένα δεδομένα), ενώ χαμηλότερες τιμές αντιστοιχούν σε πιο απλά ή προβλέψιμα δεδομένα.

ΜΕΡΟΣ Γ: Αυθεντικοποίηση Δύο Παραγόντων (2FA)

Στο τρίτο μέρος της άσκησης, καλείστε να ενσωματώσετε έναν μηχανισμό αυθεντικοποίησης δύο παραγόντων, με χρήση One-Time Password (OTP), για την προστασία κρίσιμων λειτουργιών.

Βήματα Υλοποίησης

- Αρχικά, το πρόγραμμα θα δημιουργεί ένα μυστικό κλειδί (secret key), το οποίο θα χρησιμοποιείται για την παραγωγή των OTP.
- Στη συνέχεια, θα παράγεται ένας δυναμικός κωδικός μιας χρήσης (OTP), ο οποίος θα εμφανίζεται στον χρήστη ή θα υπολογίζεται μέσω βιβλιοθήκης.



- Έπειτα, το πρόγραμμα θα ζητά από τον χρήστη να εισάγει τον κωδικό που δημιουργήθηκε.
- Στο επόμενο βήμα, θα γίνεται επαλήθευση του OTP. Αν ο κωδικός είναι σωστός, η πρόσβαση θα επιτρέπεται και το πρόγραμμα θα συνεχίζει στη ζητούμενη λειτουργία. Αν είναι λανθασμένος, η πρόσβαση θα απορρίπτεται.
- Τέλος, ο μηχανισμός αυτός θα πρέπει να χρησιμοποιείται πριν από την εκτέλεση κρίσιμων ενεργειών, όπως ο έλεγχος ακεραιότητας αρχείου.

Μέρος Δ: Ενσωμάτωση Salt

Στο τέταρτο και τελευταίο μέρος της άσκησης, καλείστε να επεκτείνετε την υλοποίηση του hashing, ενσωματώνοντας τη χρήση **salt**.

Το salt είναι μία τυχαία τιμή που προστίθεται στα δεδομένα πριν τον υπολογισμό του hash, αυξάνοντας σημαντικά την ασφάλεια.

Βήματα Υλοποίησης

- Αρχικά, το πρόγραμμα θα δημιουργεί ένα τυχαίο salt (π.χ. 16 bytes).
- Στη συνέχεια, το salt θα συνδυάζεται με τα δεδομένα του αρχείου πριν τον υπολογισμό του hash (π.χ. salt + data).
- Έπειτα, θα υπολογίζεται το νέο hash με βάση τα salted δεδομένα.
- Στο επόμενο βήμα, το πρόγραμμα θα αποθηκεύει τόσο το hash όσο και το salt.
- Τέλος, στον έλεγχο ακεραιότητας, το πρόγραμμα θα χρησιμοποιεί το αποθηκευμένο salt για να επαναυπολογίσει το hash και να το συγκρίνει με το αρχικό.

Διαδραστικό Μενού

Το πρόγραμμα θα πρέπει να ενσωματώνει όλες τις παραπάνω λειτουργίες σε ένα ενιαίο διαδραστικό μενού, μέσω του οποίου ο χρήστης θα μπορεί να επιλέγει ποια λειτουργία επιθυμεί να εκτελέσει, όπως υπολογισμό hash, έλεγχο ακεραιότητας, υπολογισμό εντροπίας και αυθεντικοποίηση.

Παραδοτέα

Να παραδώσετε τον πηγαίο κώδικα της εφαρμογής σε Python, καθώς και μία αναφορά στην οποία θα περιγράφεται η υλοποίηση, θα παρουσιάζονται παραδείγματα εκτέλεσης και θα γίνεται ανάλυση των αποτελεσμάτων, ιδιαίτερα όσον αφορά την εντροπία.



Παράρτημα-Υποδείξεις

Ανάγνωση Αρχείου

Θα πρέπει να υλοποιήσετε λειτουργία που:

- Δέχεται όνομα αρχείου από τον χρήστη
- Διαβάζει το αρχείο σε binary μορφή

Υπολογισμός Hash

Να υλοποιηθεί υπολογισμός των παρακάτω συναρτήσεων:

- MD5
- SHA-1 ή SHA-256
- SHA-256 (όπως χρησιμοποιείται σε blockchain εφαρμογές)
- SHA-3 (Keccak)

Ο χρήστης θα μπορεί να επιλέξει:

- έναν αλγόριθμο ή
- όλους τους διαθέσιμους

Τα αποτελέσματα θα εμφανίζονται στην οθόνη.

Αποθήκευση Hash

Το πρόγραμμα θα:

- Αποθηκεύει το hash σε αρχείο (π.χ. filename.hash)
- Συνδέει το hash με το αρχικό αρχείο

Έλεγχος Ακεραιότητας

Θα υλοποιήσετε λειτουργία που:

1. Υπολογίζει εκ νέου το hash του αρχείου
2. Συγκρίνει με το αποθηκευμένο hash

Αναμενόμενη Έξοδος:

- Το αρχείο δεν έχει τροποποιηθεί
- Το αρχείο έχει αλλοιωθεί



Υπολογισμός Εντροπίας

Η εντροπία υπολογίζεται ως:

$$H = - \sum_{i=1}^{256} p(x_i) \log_2 p(x_i)$$

όπου:

- x_i : η i-οστή τιμή byte (0 έως 255)
- $p(x_i)$: πιθανότητα εμφάνισης του byte στο αρχείο

Τι σημαίνει πρακτικά

Πρέπει να:

1. Διαβάσει το αρχείο σε binary μορφή
2. Μετρήσει πόσες φορές εμφανίζεται κάθε byte (0–255)
3. Υπολογίσει πιθανότητες:

$p(x) = \text{εμφανίσεις byte} / \text{συνολικό πλήθος bytes}$

4. Εφαρμόσει τον τύπο

Σημαντική Διευκρίνιση

- Δεν υπολογίζουμε εντροπία “χαρακτήρων” (εκτός αν είναι text μόνο)
- Υπολογίζουμε εντροπία **σε επίπεδο byte**
- Αυτό κάνει τη μέθοδο γενική για οποιοδήποτε αρχείο (pdf, εικόνα, exe κτλ.)

Ερώτηση για την Αναφορά

- Γιατί ένα encrypted αρχείο έχει entropy κοντά στο 8
- Γιατί ένα απλό text αρχείο έχει μικρότερη

Διαδραστικό Μενού

Το πρόγραμμα θα πρέπει να περιλαμβάνει μενού επιλογών:

1. Υπολογισμός Hash
2. Έλεγχος Ακεραιότητας
3. Υπολογισμός Εντροπίας
4. 2FA Authentication
5. Έξοδος